SIMATS ENGINEERING

ASSESSMENT TOOL 1 – High (Coding Challenge)

COURSE CODE: CSA09 **COURSE NAME: Programming in Java**

COURSE OUTCOME COVERED

CO1: *Apply object-oriented programming principles such as classes, objects, encapsulation, data hiding, inheritance, and polymorphism to design and implement entity manipulations (BL3,BL4)*

QUESTIONS: 10 **Total Marks: 100**

ANNEXURE A **Coding Challenge**

Code of Conduct:

I N. Hemanth Reddy (192472215)_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases

Coding Challenge

1. Library Book Management using Classes & Encapsulation

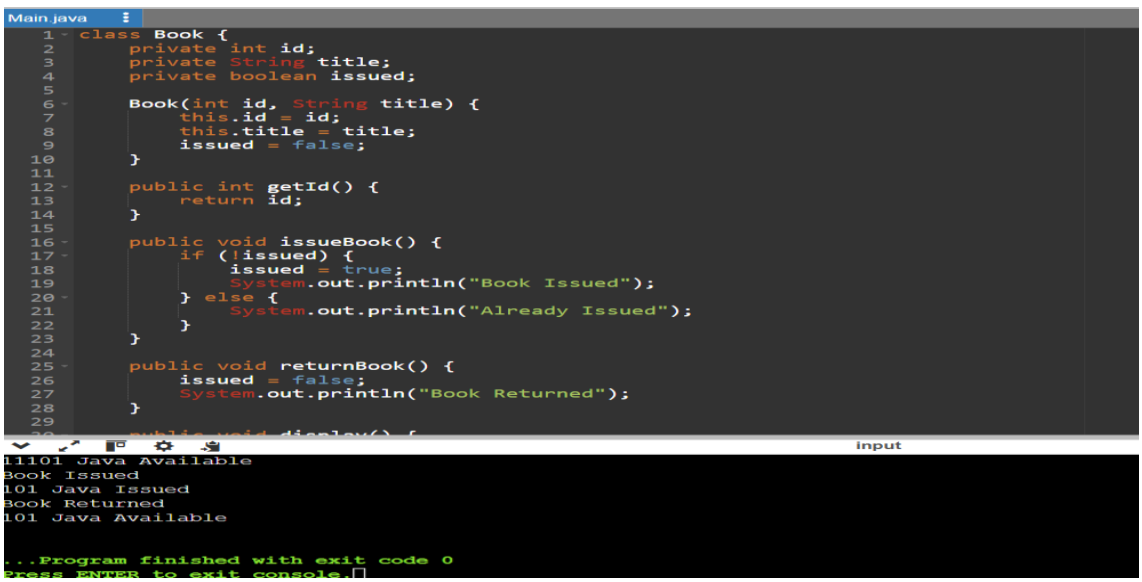
Design a Book class with private fields such as bookId, title, author, and price. Implement constructors, getters, setters, and methods to issue and return books. Create a menu-driven program to manage multiple books using an array of objects. Also, construct suitable test cases to validate book addition, issue, return, and search operations. BTL: BL3 (Apply)\

CODE;

```
class Book {
    private int id;
    private String title;
    private boolean issued;
    Book(int id, String title) {
        this.id = id;
        this.title = title;
        issued = false;
    }
    public int getId() {
        return id;
    }
    public void issueBook() {
        if (!issued) {
            issued = true;
            System.out.println("Book Issued");
        } else {
            System.out.println("Already Issued");
        }
    }


    public void returnBook() {
```

```
        issued = false;
        System.out.println("Book Returned");
    }
    public void display() {
        System.out.println(id + " " + title + " " + (issued ? "Issued" : "Available"));
    }
}
public class Main {
    public static void main(String[] args) {
        Book[] books = new Book[2];
        books[0] = new Book(101, "Java");
        books[1] = new Book(102, "Python");
        books[0].display();
        books[0].issueBook();
        books[0].display();
        books[0].returnBook();
        books[0].display();
    }
}
```

Output:



2. Employee Payroll System using Inheritance

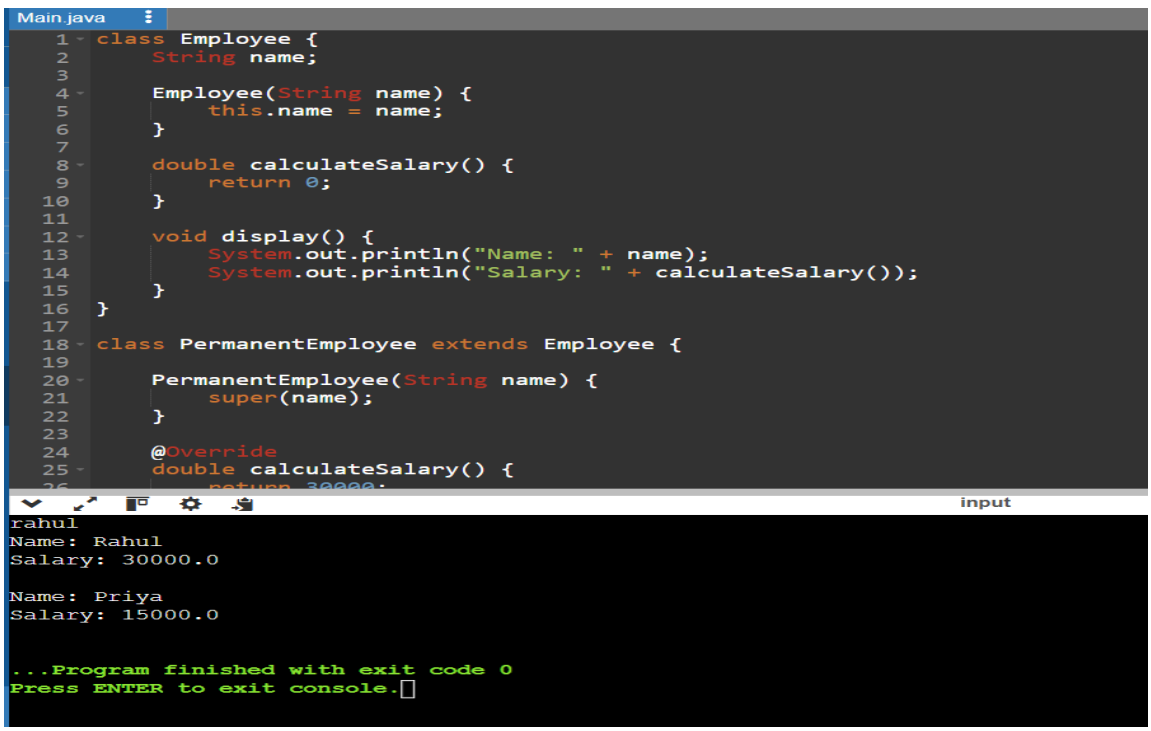
Design a payroll system with a base class Employee and derived classes PermanentEmployee and ContractEmployee. Override a method calculateSalary() in each subclass to compute salary differently. Display employee details and salary using runtime polymorphism. Also, construct suitable test cases to validate salary calculation for different employee types. BTL: BL3 (Apply)

CODE:

```
class Employee {
    String name;
    Employee(String name) {
        this.name = name;
    }
    double calculateSalary() {
        return 0;
    }
    void display() {
        System.out.println("Name: " + name);
        System.out.println("Salary: " + calculateSalary());
    }
}
class PermanentEmployee extends Employee {
    PermanentEmployee(String name) {
        super(name);
    }
    @Override
    double calculateSalary() {
        return 30000;
    }
}
class ContractEmployee extends Employee {
    ContractEmployee(String name) {
```

```
        super(name);
    }
    @Override
    double calculateSalary() {
        return 15000;
    }
}
public class Main {
    public static void main(String[] args) {
        Employee e1 = new PermanentEmployee("Rahul");
        Employee e2 = new ContractEmployee("Priya")
        e1.display();
        System.out.println();
        e2.display();
    }
}
```

Output:



3. Vehicle Rental System using Runtime Polymorphism

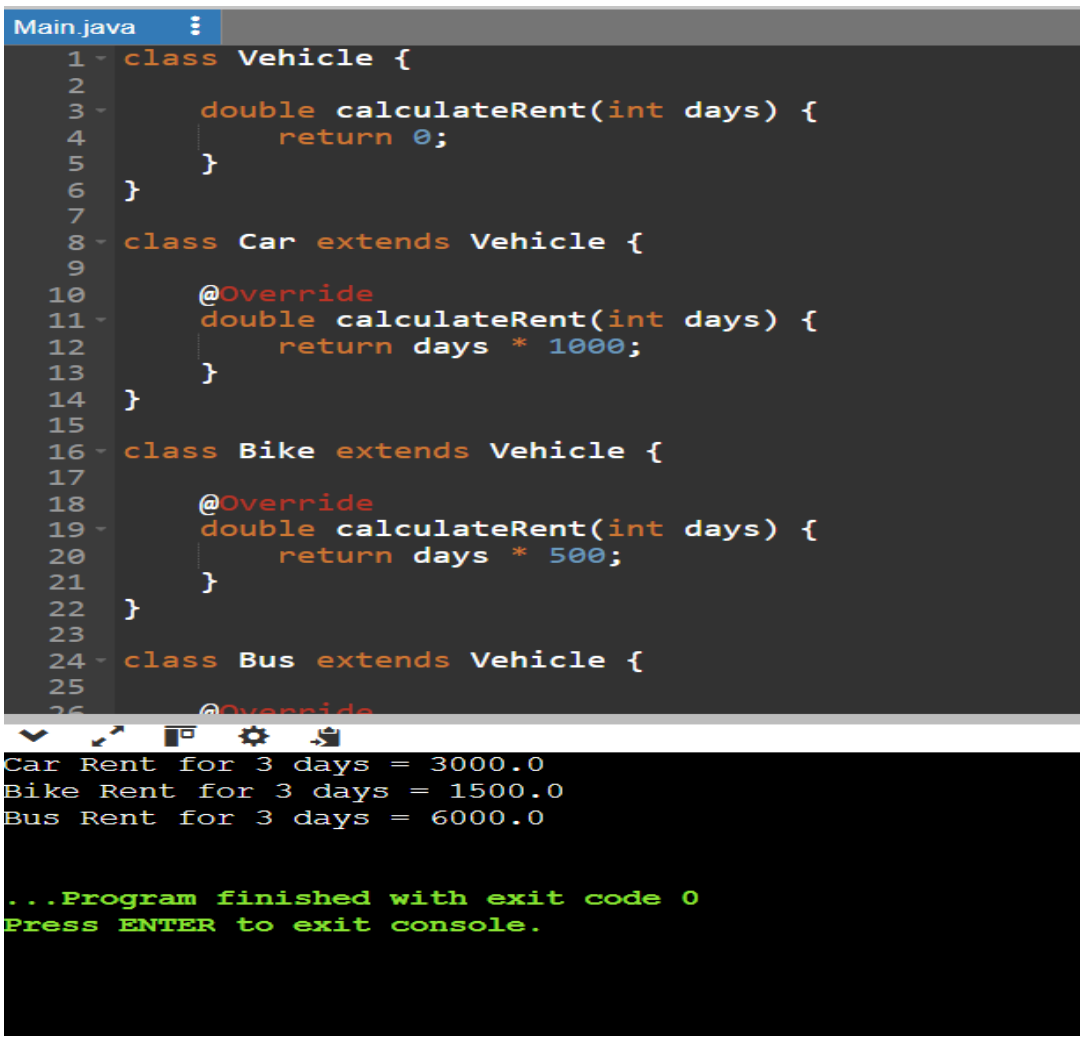
Develop a vehicle rental system with a superclass Vehicle and subclasses Car, Bike, and Bus. Override the method calculateRent(int days) in each subclass. Store vehicles in an array of Vehicle references and generate rental bills dynamically. Also, construct suitable test cases to validate rent calculation for different vehicle categories. BTL: BL4 (Analyze)

CODE:

```
class Vehicle {
    double calculateRent(int days) {
        return 0;
    }
}
class Car extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 1000;
    }
}
class Bike extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 500;
    }
}
class Bus extends Vehicle {
    @Override
    double calculateRent(int days) {
        return days * 2000;
    }
}
public class Main {
```

```
    public static void main(String[] args) {

        Vehicle v;

        v = new Car();

        System.out.println("Car Rent for 3 days = " + v.calculateRent(3));

        v = new Bike();

        System.out.println("Bike Rent for 3 days = " + v.calculateRent(3));

        v = new Bus();

        System.out.println("Bus Rent for 3 days = " + v.calculateRent(3));

    }

}
```

Output:



4. Bank Account System using Data Hiding

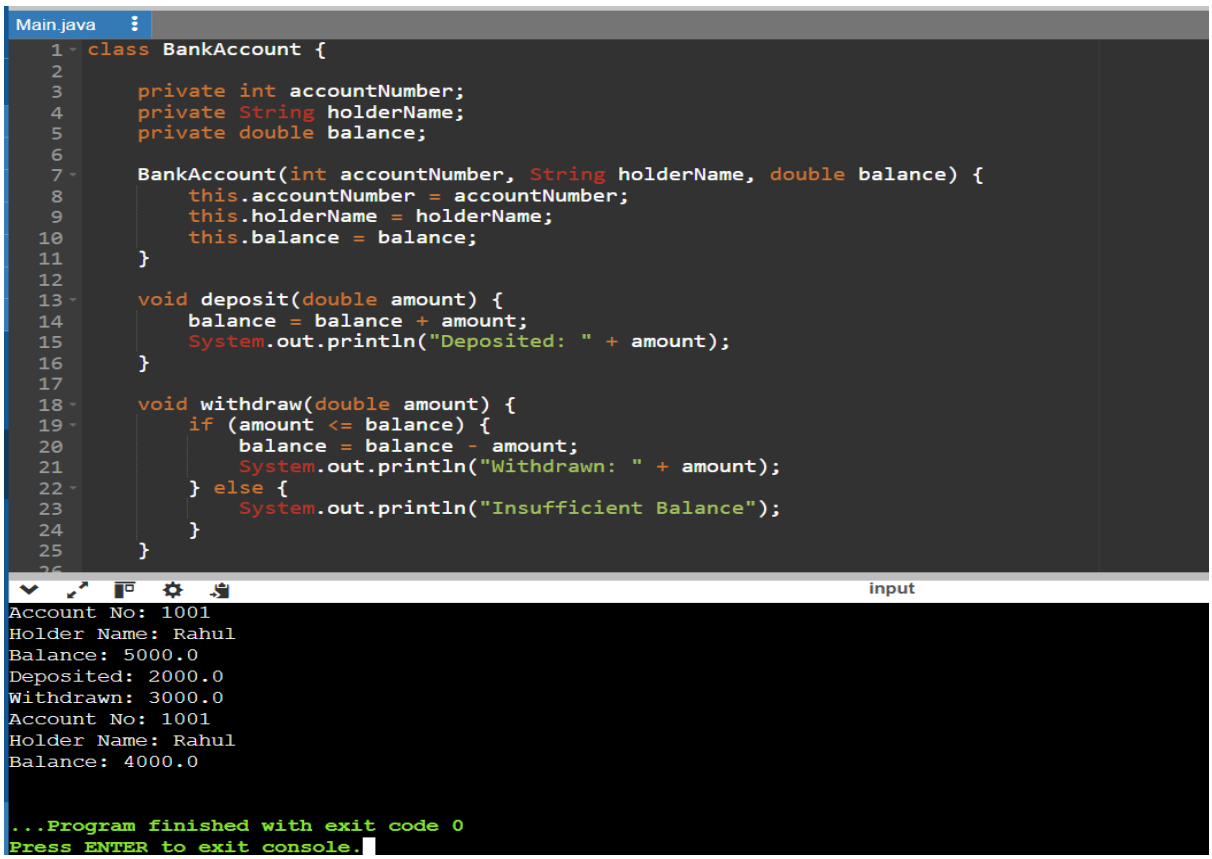
Design a BankAccount class with private fields accountNumber, holderName, and balance. Implement methods for deposit, withdrawal, and balance inquiry with proper validation. Prevent direct modification of balance from outside the class. Also, construct suitable test cases to validate valid and invalid transactions. BTL: BL3 (Apply)

Code;

```
class BankAccount {

    private int accountNumber;

    private String holderName;

    private double balance;

    BankAccount(int accountNumber, String holderName, double balance) {

        this.accountNumber = accountNumber;

        this.holderName = holderName;

        this.balance = balance;

    }

    void deposit(double amount) {

        balance = balance + amount;

        System.out.println("Deposited: " + amount);

    }

    void withdraw(double amount) {

        if (amount <= balance) {

            balance = balance - amount;

            System.out.println("Withdrawn: " + amount);

        } else {

            System.out.println("Insufficient Balance");

        }

    }

    void displayBalance() {

        System.out.println("Account No: " + accountNumber);

        System.out.println("Holder Name: " + holderName);
```

```
        System.out.println("Balance: " + balance);
    }
}
public class Main {
    public static void main(String[] args) {
        BankAccount b = new BankAccount(1001, "Rahul", 5000);
        b.displayBalance();
        b.deposit(2000);
        b.withdraw(3000);
        b.displayBalance();
    }
}
```

Output:



5. University Admission System using Method Overloading

Create an Admission class that overloads the method calculateFee() for:

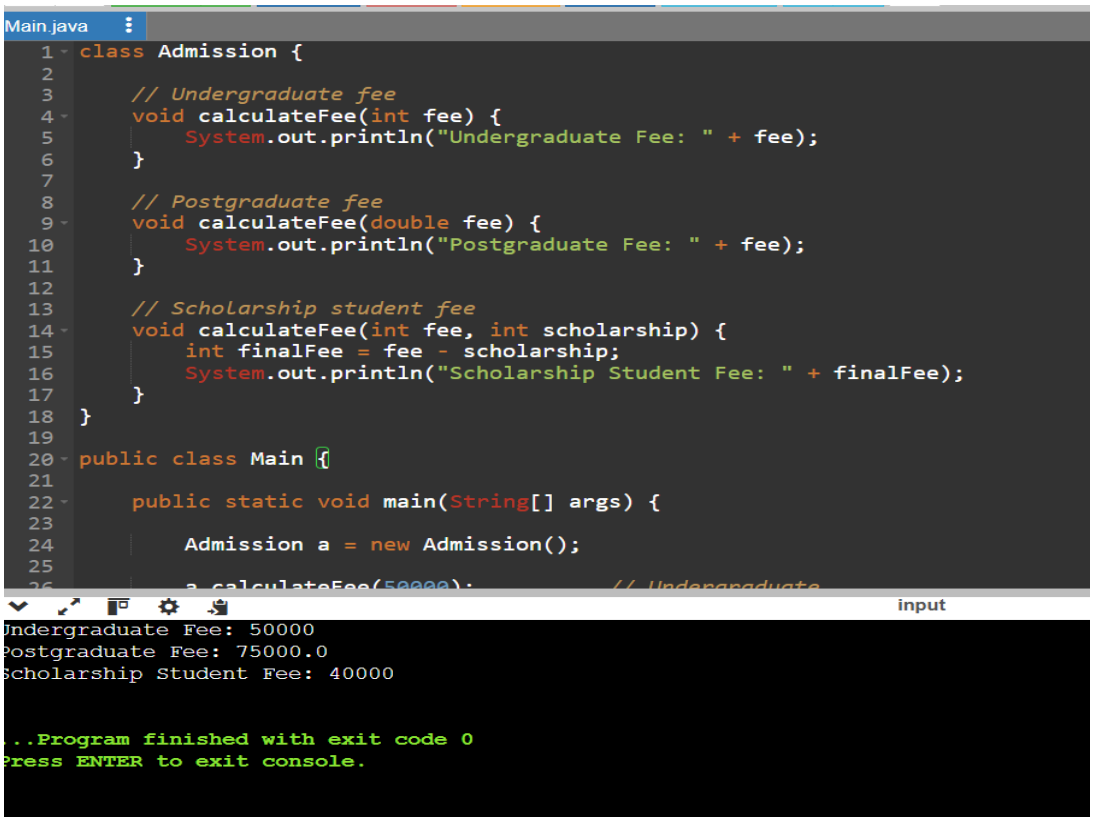
- Undergraduate students,
- Postgraduate students,
- Scholarship students.

Display the fee structure based on user input and compare the overloaded method executions. Also, construct suitable test cases to validate each overloaded version. BTL: BL3 (Apply)

Code:

```
class Admission {
    void calculateFee(int fee) {
        System.out.println("Undergraduate Fee: " + fee);
    }
    void calculateFee(double fee) {
        System.out.println("Postgraduate Fee: " + fee);
    }
    void calculateFee(int fee, int scholarship) {
        int finalFee = fee - scholarship;
        System.out.println("Scholarship Student Fee: " + finalFee);
    }
}
public class Main {
    public static void main(String[] args) {
        Admission a = new Admission();
        a.calculateFee(50000);          // Undergraduate
        a.calculateFee(75000.0);        // Postgraduate
        a.calculateFee(50000, 10000);   // Scholarship Student
    }
}
```

Output:



6. Shape Area Calculator using Abstract Class & Polymorphism

Design an abstract class Shape with an abstract method area(). Implement subclasses Circle, Rectangle, and Triangle. Use an array of Shape references to compute and display areas dynamically. Also, construct suitable test cases to validate area calculations and polymorphic behavior. BTL: BL4 (Analyze)

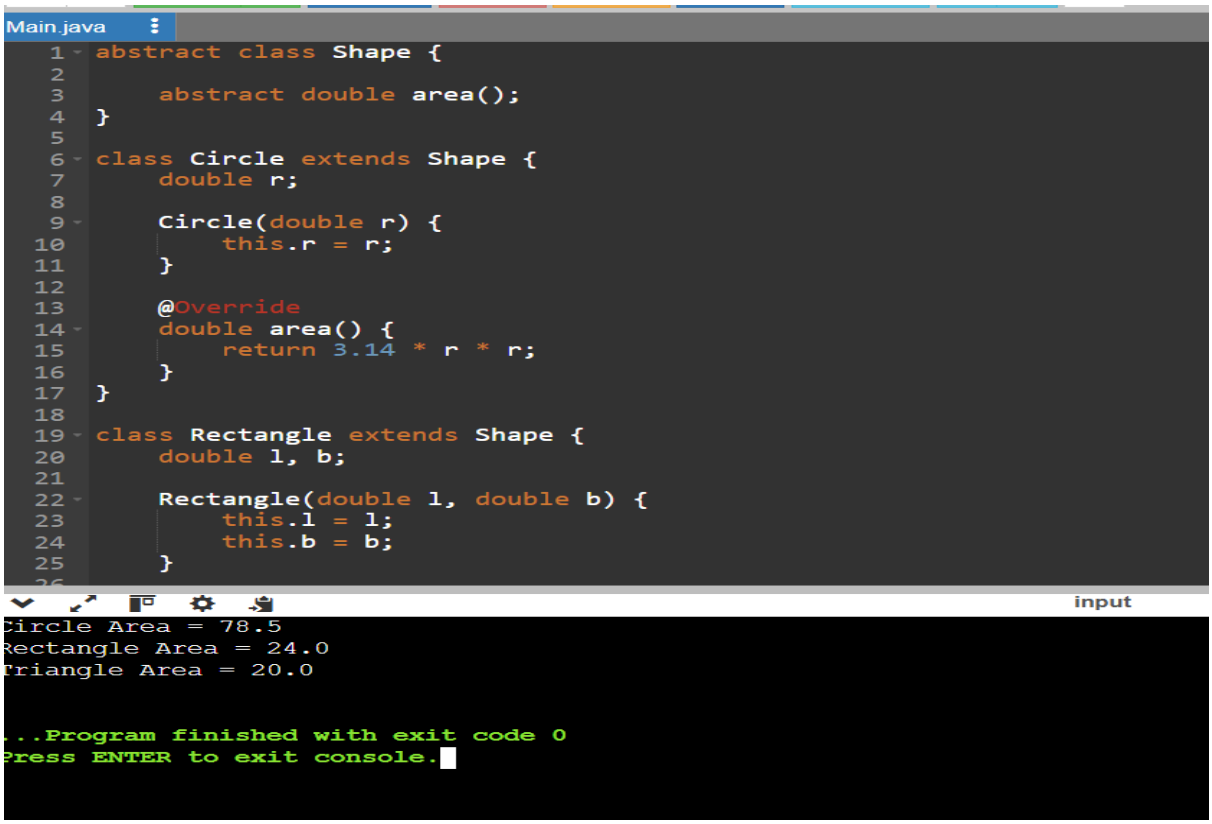
Code:

```
abstract class Shape {

    abstract double area();

}

class Circle extends Shape {

    double r;

    Circle(double r) {

        this.r = r;

    }


    @Override
```

```
    double area() {
        return 3.14 * r * r;
    }
}
class Rectangle extends Shape {
    double l, b;
    Rectangle(double l, double b) {
        this.l = l;
        this.b = b;
    }
    @Override
    double area() {
        return l * b;
    }
}
class Triangle extends Shape {
    double b, h;
    Triangle(double b, double h) {
        this.b = b;
        this.h = h;
    }
    @Override
    double area() {
        return 0.5 * b * h;
    }
}
public class Main {
    public static void main(String[] args) {
        Shape[] s = new Shape[3];
```

```
        s[0] = new Circle(5);

        s[1] = new Rectangle(4, 6);

        s[2] = new Triangle(8, 5);

        System.out.println("Circle Area = " + s[0].area());

        System.out.println("Rectangle Area = " + s[1].area());

        System.out.println("Triangle Area = " + s[2].area());

    }

}
```

Output:



7. Hospital Management System using Interfaces

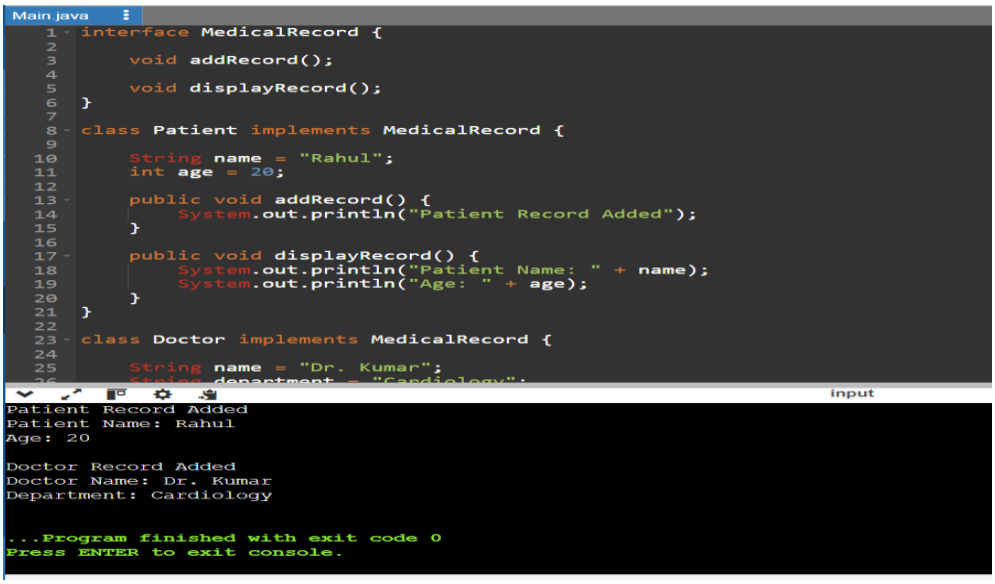
Develop a hospital management system with an interface MedicalRecord containing methods addRecord() and displayRecord(). Implement the interface in classes Patient and Doctor. Demonstrate interface-based abstraction and object interaction. Also, construct suitable test cases to validate record creation and display. BTL: BL3 (Apply)

Code:

```
interface MedicalRecord {
    void addRecord();
    void displayRecord();
}
class Patient implements MedicalRecord {
    String name = "Rahul";
    int age = 20;
    public void addRecord() {
        System.out.println("Patient Record Added");
    }
    public void displayRecord() {
        System.out.println("Patient Name: " + name);
        System.out.println("Age: " + age);
    }
}
class Doctor implements MedicalRecord {
    String name = "Dr. Kumar";
    String department = "Cardiology";
    public void addRecord() {
        System.out.println("Doctor Record Added");
    }
    public void displayRecord() {
        System.out.println("Doctor Name: " + name);
        System.out.println("Department: " + department);
```

```
    }
}
public class Main {
    public static void main(String[] args) {
        MedicalRecord p = new Patient();
        MedicalRecord d = new Doctor();
        p.addRecord();
        p.displayRecord();
        System.out.println();
        d.addRecord();
        d.displayRecord();
    }
}
```

Output:



8. Online Shopping Order System using Packages

Create two packages:

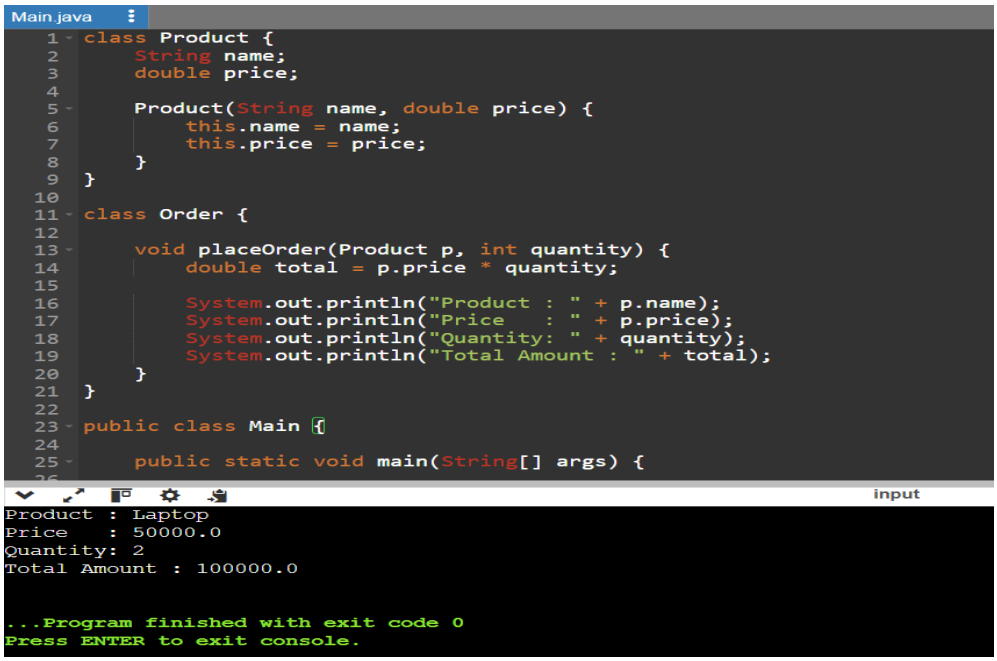
- shopping.product
- shopping.order

Implement classes Product and Order in separate packages. Import the packages into a main application that places orders and calculates the total amount. Also, construct suitable test cases to validate package usage, order placement, and bill generation. BTL: BL3 (Apply)

Code:

```
class Product {
    String name;
    double price;
    Product(String name, double price) {
        this.name = name;
        this.price = price;
    }
}
class Order {
    void placeOrder(Product p, int quantity) {
        double total = p.price * quantity;
        System.out.println("Product : " + p.name);
        System.out.println("Price   : " + p.price);
        System.out.println("Quantity: " + quantity);
        System.out.println("Total Amount : " + total);
    }
}
public class Main {
    public static void main(String[] args) {
        Product p = new Product("Laptop", 50000);
        Order o = new Order();
        o.placeOrder(p, 2);
    }
}
```

Output:



9. Student Result Analyzer using Arrays of Objects

Design a Student class containing roll number, name, and marks in three subjects. Store multiple student objects in an array and implement methods to calculate total, average, grade, and class topper. Also, construct suitable test cases to validate grading and topper identification. BTL: BL3 (Apply)

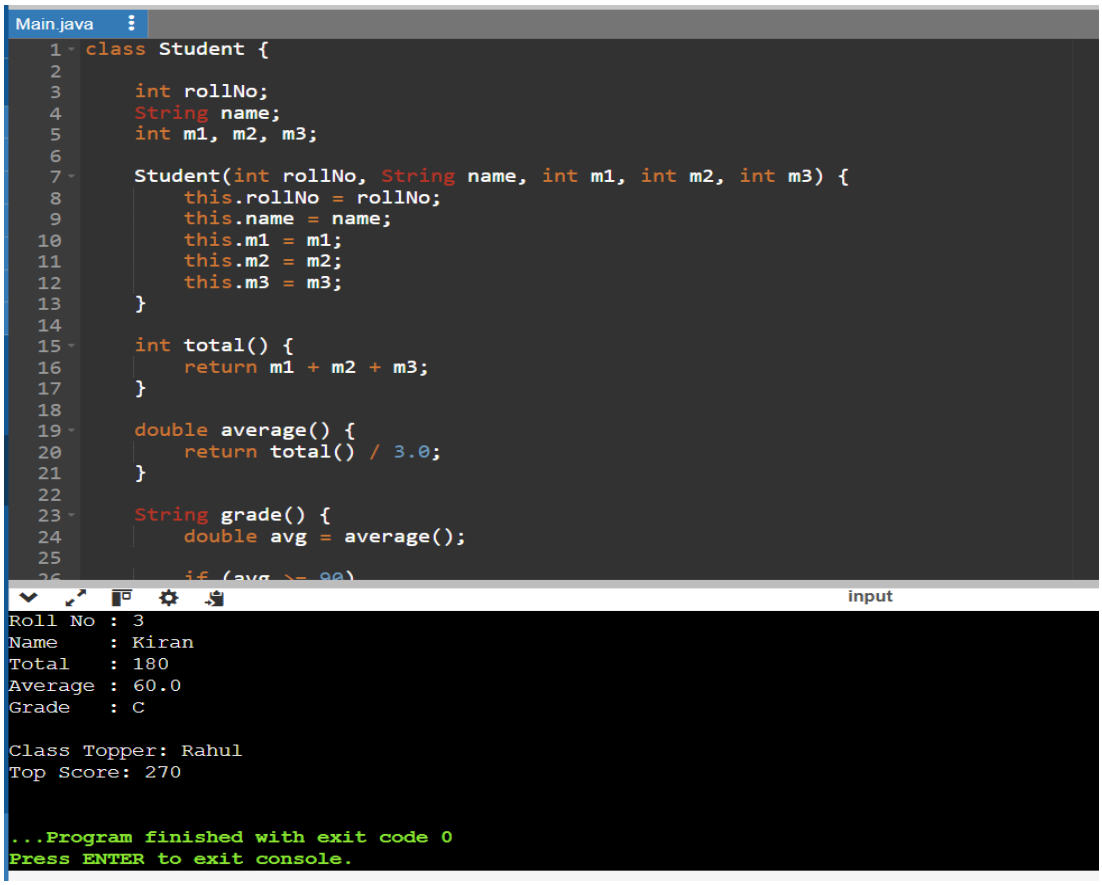
Code:

```
class Student {

    int rollNo;

    String name;

    int m1, m2, m3;

    Student(int rollNo, String name, int m1, int m2, int m3) {

        this.rollNo = rollNo;

        this.name = name;

        this.m1 = m1;

        this.m2 = m2;

        this.m3 = m3;

    }

    int total() {

        return m1 + m2 + m3;
```

```
    }
    double average() {
        return total() / 3.0;
    }
    String grade() {
        double avg = average();
        if (avg >= 90)
            return "A";
        else if (avg >= 75)
            return "B";
        else if (avg >= 50)
            return "C";
        else
            return "Fail";
    }
}
public class Main {
    public static void main(String[] args) {
        Student[] s = {
            new Student(1, "Rahul", 90, 85, 95),
            new Student(2, "Priya", 80, 75, 70),
            new Student(3, "Kiran", 60, 65, 55)
        };
        int max = 0;
        String topper = "";
        for (Student st : s) {
            System.out.println("Roll No : " + st.rollNo);
            System.out.println("Name    : " + st.name);
            System.out.println("Total   : " + st.total());
```

```
            System.out.println("Average : " + st.average());
            System.out.println("Grade   : " + st.grade());
            System.out.println();
            if (st.total() > max) {
                max = st.total();
                topper = st.name;
            }
        }
        System.out.println("Class Topper: " + topper);
        System.out.println("Top Score: " + max);
    }
}
```

Output:



10. Smart Home Device Controller using Hierarchical Inheritance

Develop a smart home system with a base class Device and derived classes Light, Fan, and AirConditioner. Implement methods to turn devices on/off and display power consumption. Use polymorphism to control devices through a common reference. Also, construct suitable test cases to validate device operations and power calculations. BTL: BL4 (Analyze)

Code:

```
class Device {
    void turnOn() {
        System.out.println("Device is ON");
    }
    void turnOff() {
        System.out.println("Device is OFF");
    }
    double powerConsumption() {
        return 0;
    }
}
class Light extends Device {
    @Override
    double powerConsumption() {
        return 20;
    }
}
class Fan extends Device {
    @Override
    double powerConsumption() {
        return 75;
    }
}
class AirConditioner extends Device {
    @Override
    double powerConsumption() {
        return 1500;
    }
```

```
}
public class Main {
    public static void main(String[] args) {
        Device d;
        d = new Light();
        d.turnOn();
        System.out.println("Light Power: " + d.powerConsumption() + " Watts");
        d.turnOff();
        System.out.println();
        d = new Fan();
        d.turnOn();
        System.out.println("Fan Power: " + d.powerConsumption() + " Watts");
        d.turnOff();
        System.out.println();
        d = new AirConditioner();
        d.turnOn();
        System.out.println("AC Power: " + d.powerConsumption() + " Watts");
        d.turnOff();
    }
}
```

Output:

Main.java

```java
class Device {

    void turnOn() {
        System.out.println("Device is ON");
    }

    void turnOff() {
        System.out.println("Device is OFF");
    }

    double powerConsumption() {
        return 0;
    }
}

class Light extends Device {

    @Override
    double powerConsumption() {
        return 20;
    }
}

class Fan extends Device {

    @Override
```

input

```
Device is ON
Fan Power: 75.0 Watts
Device is OFF

Device is ON
AC Power: 1500.0 Watts
Device is OFF


...Program finished with exit code 0
Press ENTER to exit console.
```